

OnR.R

shariqueshafi

2025-11-26

```
library(readxl)
library(dplyr)
```

```
##
## Attaching package: 'dplyr'
```

```
## The following objects are masked from 'package:stats':
##
##   filter, lag
```

```
## The following objects are masked from 'package:base':
##
##   intersect, setdiff, setequal, union
```

```
library(lubridate)
```

```
##
## Attaching package: 'lubridate'
```

```
## The following objects are masked from 'package:base':
##
##   date, intersect, setdiff, union
```

```
setwd("~/Desktop/MLWBA")

OnR = read_excel("Online Retail.xlsx")
summary(OnR)
```

```
## InvoiceNo StockCode Description Quantity
## Length:541909 Length:541909 Length:541909 Min. :-80995.000
## Class :character Class :character Class :character 1st Qu.: 1.000
## Mode :character Mode :character Mode :character Median : 3.000
## Mean : 9.552
## 3rd Qu.: 10.000
## Max. : 80995.000
##
## InvoiceDate UnitPrice CustomerID
## Min. :2010-12-01 08:26:00 Min. : -11062.060 Min. :12346
## 1st Qu.:2011-03-28 11:34:00 1st Qu.: 1.250 1st Qu.:13953
## Median :2011-07-19 17:17:00 Median : 2.080 Median :15152
## Mean :2011-07-04 13:34:57 Mean : 4.611 Mean :15288
## 3rd Qu.:2011-10-19 11:27:00 3rd Qu.: 4.130 3rd Qu.:16791
## Max. :2011-12-09 12:50:00 Max. : 38970.000 Max. :18287
## NA's :135080
##
## Country
## Length:541909
## Class :character
## Mode :character
##
##
##
##
```

```
## To check duplicate
sum(duplicated(OnR))
```

```
## [1] 5268
```

```
sum(OnR$Quantity<0)
```

```
## [1] 10624
```

```
sum(OnR$UnitPrice<0)
```

```
## [1] 2
```

```
## Clean the data
OnR = OnR %>% distinct() %>% filter(
  !is.na(CustomerID), Quantity > 0, UnitPrice > 0,
)

OnR$TotalAmount = OnR$Quantity * OnR$UnitPrice
OnR$InvoiceDate = as.POSIXct(OnR$InvoiceDate,tz="UTC")

## RFM data
reference_date = as.Date(max(OnR$InvoiceDate)) +1
rfm = OnR %>% group_by(CustomerID) %>%
  summarise(
    Recency = as.integer(difftime(reference_date,
                                   as.Date(max(InvoiceDate)),
                                   units = "days")),
    Frequency = n_distinct(InvoiceNo),
    Monetary = sum(TotalAmount),
    .groups = "drop"
  )
View(rfm)
summary(rfm)
```

```
##      CustomerID      Recency      Frequency      Monetary
## Min.      :12346 Min.      : 1.00 Min.      : 1.000 Min.      :    3.75
## 1st Qu.:13813 1st Qu.: 18.00 1st Qu.: 1.000 1st Qu.:   306.48
## Median :15300 Median : 51.00 Median : 2.000 Median :   668.57
## Mean    :15300 Mean    : 93.06 Mean    : 4.272 Mean    : 2048.69
## 3rd Qu.:16779 3rd Qu.:142.75 3rd Qu.: 5.000 3rd Qu.: 1660.60
## Max.    :18287 Max.    :374.00 Max.    :209.000 Max.    :280206.02
```

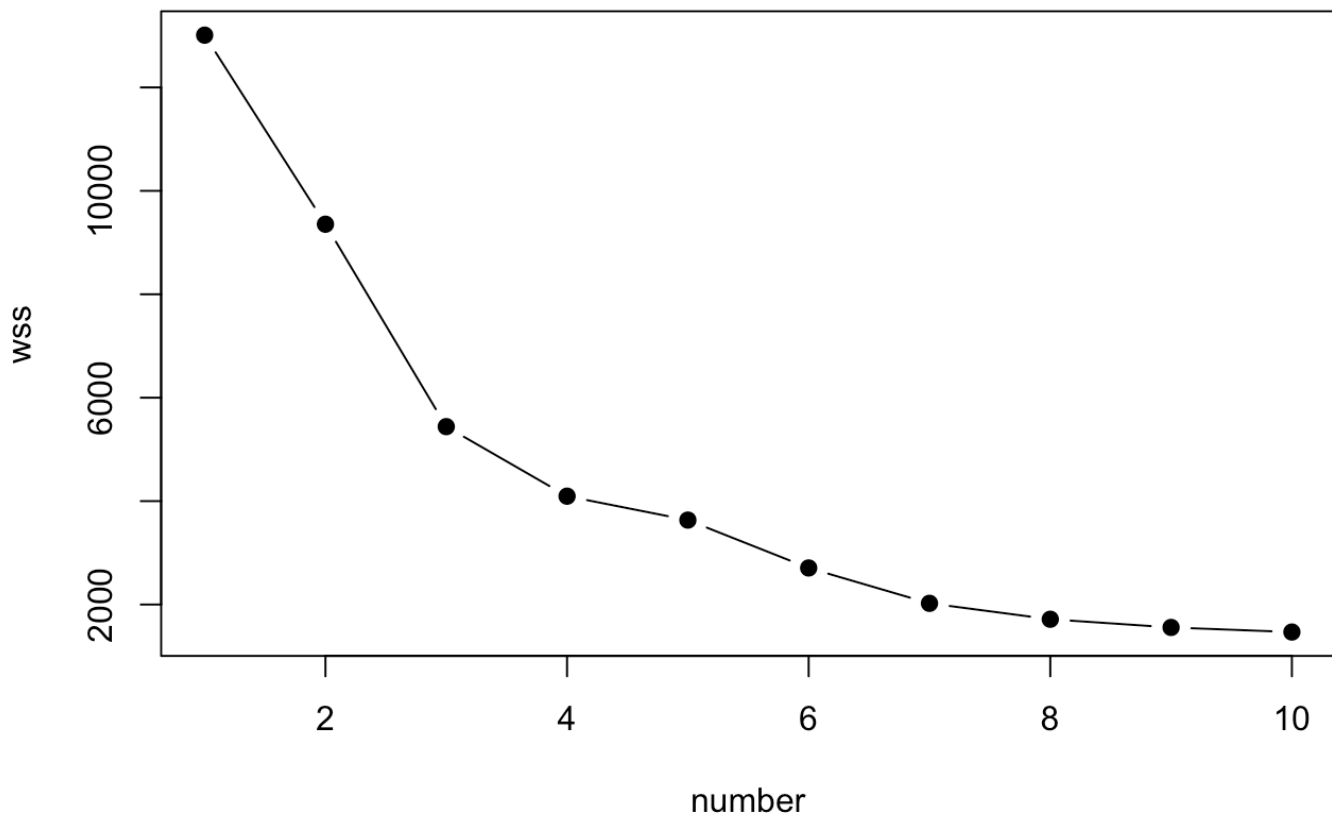
```
rfm_mat = as.matrix(rfm[,c("Recency", "Frequency", "Monetary")])
summary(rfm)
```

```
##      CustomerID      Recency      Frequency      Monetary
## Min.      :12346 Min.      : 1.00 Min.      : 1.000 Min.      :    3.75
## 1st Qu.:13813 1st Qu.: 18.00 1st Qu.: 1.000 1st Qu.:   306.48
## Median :15300 Median : 51.00 Median : 2.000 Median :   668.57
## Mean    :15300 Mean    : 93.06 Mean    : 4.272 Mean    : 2048.69
## 3rd Qu.:16779 3rd Qu.:142.75 3rd Qu.: 5.000 3rd Qu.: 1660.60
## Max.    :18287 Max.    :374.00 Max.    :209.000 Max.    :280206.02
```

```
rfm_scaled = scale(rfm_mat)

### K Means on RFM Analysis ###
number = 1:10
wss = 1:10

for(i in 1:10)
{
  wss[i] = kmeans (rfm_scaled,i)$tot.withinss
}
plot(number,wss,type="b",pch=19)
```



```
### K means at cluster 4 ###
set.seed(42)
km = kmeans(rfm_scaled,4)
accuracy = km$betweenss/km$totss
accuracy
```

```
## [1] 0.6852179
```

```
km = kmeans(rfm_scaled,5)
accuracy = km$betweenss/km$totss
accuracy
```

```
## [1] 0.7207605
```

```
###Profile the clusters by aggregating data
# The aggregate() function calculates the mean of each variable for each cluster group
> cluster_profiles <- aggregate(select(iris, -cluster), by = list(cluster = iris$cluster), mean)
```

```
str(km)
```

```
# View the profiles
> print(cluster_profiles)
```

```
## List of 9
## $ cluster      : int [1:4338] 4 3 3 3 5 3 1 5 1 3 ...
## $ centers      : num [1:5, 1:3] 0.605 -0.854 -0.586 -0.771 2.018 ...
## ..- attr(*, "dimnames")=List of 2
## .. ..$ : chr [1:5] "1" "2" "3" "4" ...
## .. ..$ : chr [1:3] "Recency" "Frequency" "Monetary"
## $ totss       : num 13011
## $ withinss    : num [1:5] 157 1758 659 924 135
## $ tot.withinss: num 3633
## $ betweenss   : num 9378
## $ size        : int [1:5] 806 13 2697 199 623
## $ iter        : int 4
## $ ifault      : int 0
## - attr(*, "class")= chr "kmeans"
```

```
rfm$cluster = km$cluster
```

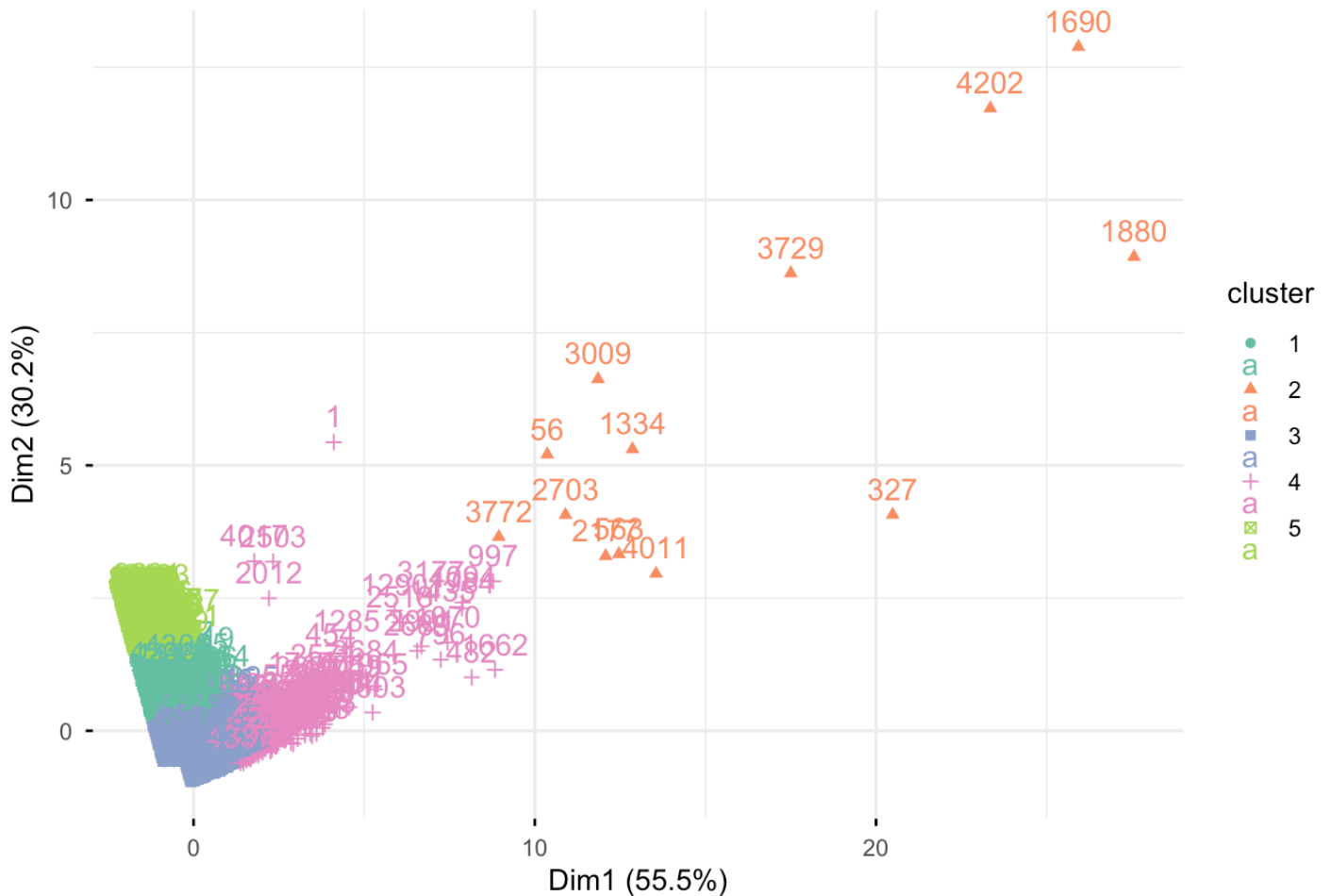
```
#### Cluster Plot ####
library(factoextra)
```

```
## Loading required package: ggplot2
```

```
## Welcome! Want to learn more? See two factoextra-related books at https://goo.gl/ve3WBa
```

```
fviz_cluster(
  object = km,
  data = rfm_scaled,
  gemo = "point",
  show.clust.cent = FALSE,
  ellipse = FALSE,
  ggtheme = theme_minimal(),
  palette = "Set2",
  main = "RFM Cluster"
)
```

RFM Cluster



```
### Profiling of RFM Cluster ###
table(rfm$cluster)
```

```
##
##      1      2      3      4      5
## 806    13 2697    199    623
```

```
cmeans = aggregate(rfm[c("Recency","Frequency","Monetary")],
                    by= list(Cluster=rfm$cluster),
                    FUN=mean)

cmeans
```

```
##      Cluster      Recency Frequency      Monetary
## 1          1 153.606700  2.093052    697.0399
## 2          2   7.615385 82.538462 127187.9592
## 3          3 34.453096  3.881720   1435.6366
## 4          4 15.909548 22.577889 12803.8696
## 5          5 294.863563  1.300161   404.6055
```

```
churn_threshold = quantile(rfm$Recency,0.75)
churn_threshold
```

```
##      75%  
## 142.75
```

```
rfm$Churn_Flag = ifelse(rfm$Recency > churn_threshold,1,0)  
churn_by_cluster = rfm %>%  
  group_by(cluster) %>%  
  summarise(  
    Customer=n(),  
    Churners = sum(Churn_Flag),  
    Churn_Rate = round(Churners/Customer,3),  
    .groups = "drop"  
  )  
  
print(churn_by_cluster)
```

```
## # A tibble: 5 × 4  
##   cluster Customer Churners Churn_Rate  
##   <int>     <int>   <dbl>     <dbl>  
## 1         1      806     458     0.568  
## 2         2       13        0        0  
## 3         3     2697        0        0  
## 4         4      199         4     0.02  
## 5         5      623     623        1
```

```
View(churn_by_cluster)
```